

Mira Genesis IV: Bounded Recursive Process Improvement Across Two Successive Generations

Prospective Replay, External Evaluation,
and a $G0 \rightarrow G1 \rightarrow G2$ Chain

Anthony Mets

Independent Researcher

<https://github.com/mjodheim/mira-genesis>

Research snapshot: 23 September 2026

Preprint DOI: pending

Abstract

Recursive self-improvement claims are easy to overstate because improvements in a search process, a task solver, an evaluator, and a model’s underlying capabilities are different phenomena. We report a bounded, prospectively validated two-transition process-improvement chain in MIRA GENESIS, a persistent software-lineage experiment whose editable search policy is separated from a fixed external organism evaluator.

The chain begins with an offline meta-search result, V14, that produces a first child generation G1. In the subsequent real-organism process experiment, V15.4, G1 reaches the same retained champion quality as G0 while consuming two hidden-evaluator observations instead of four, yielding frozen lexicographic utilities $G0=(0,0,-4)$ and $G1=(0,0,-2)$. Attempts to continue recursively then fail in V16 and V17, and V18 exposes a specific overfitting shortcut: four independently proposed executable children all improve the public replay surface but tie G1 on all 24 fresh holdout worlds.

V19 responds prospectively by expanding development to a stratified public corpus of 13 real replay worlds and 188 synthetic worlds, while forbidding direct access to world identity. Four isolated proposer attempts converge to decision-equivalent policies. The selected executable G2 is frozen before a new run-ID-derived holdout exists. On 24 fresh worlds, G2 records 24 wins, 0 losses and 0 ties against G1, improving mean replay core by 75.25 while reducing represented requests by 1.0417. V20 then applies G1 and G2 to the same reconstructed real A016 organism state. G2 prospectively requests no hidden evaluation; G1 requests two physical evaluations, both of which are neutral at quality 953. Terminal utilities are therefore $G1=(0,0,-2)$ and $G2=(0,0,0)$.

Under the project’s preregistered definition, these results establish a bounded *L3-P* recursive process-improvement chain across $G0 \rightarrow G1 \rightarrow G2$. They do not establish recursive quality uplift at equal budget, transfer to a new evaluator or organism family, fully autonomous model-only self-authorship, or open-ended recursive self-improvement.

Claim in one sentence

Under prospectively frozen rules, a lineage-mediated search policy improved twice across successive generations: first by reducing hidden evaluation cost at unchanged retained quality, then by producing a G2 policy that generalized on an unseen replay holdout and matched G1’s real-organism outcome with zero hidden evaluations instead of two.

Evidence snapshot

Stage	Parent	Child	Frozen outcome
V14 offline	G0	G1	1 future win, 0 losses, 1 adoption
V15.4 real process	G0	G1	(0,0,-4) vs (0,0,-2)
V18 holdout	G1	candidate G2	0 wins, 0 losses, 24 ties
V19 holdout	G1	admitted G2	24 wins, 0 losses, 0 ties
V20 real process	G1	G2	(0,0,-2) vs (0,0,0)

1 Introduction

Self-improving AI systems now routinely modify prompts, tools, memory, control flow, code, and other parts of their surrounding harness. Several recent systems explicitly frame this as recursive self-improvement or open-ended evolution: the Darwin Gödel Machine maintains an archive of self-modified coding agents [7]; Gödel Agent exposes agent logic to self-referential modification [5]; Group-Evolving Agents shares experience across branches [3]; Mendel Gödel Machine introduces comparative cross-lineage mutation [2]; and RRSI regularizes harness evolution to reduce benchmark-specific overfitting [4]. Dream-RSI makes the exploration policy itself the recursive target and reuses accumulated discovery trees as replay simulators [8].

These systems motivate an important measurement question. When a later generation is better, what exactly improved? It may solve more tasks, use less evaluation compute, explore more efficiently, produce better descendants, or simply fit the development benchmark more closely. These outcomes are scientifically different. A recursive process can improve without increasing the best observed task score; conversely, a score increase need not show that the machinery producing future improvements has improved.

This paper isolates one narrower phenomenon: *bounded recursive process improvement*. The target is the policy that decides which previously discovered organism state to expand, how many expansions to request, and when to stop. The task-level mutation operator is external and fixed. A hidden evaluator remains outside the editable search policy. The central test asks whether a later generation can preserve the same best retained organism outcome while consuming fewer expensive hidden-evaluator observations, and whether that process-level improvement can itself recur across a second lineage transition.

The result is deliberately weaker than an intelligence-explosion narrative. G0 and G1 both retain quality 953 in the first real-organism comparison, while G1 spends fewer observations. G2 later passes a fresh 24-world replay holdout, then stops before requesting any hidden evaluation on the real A016 state; G1 spends two evaluations and finds nothing better. The improvement is therefore in *search allocation and stopping*, not in a higher real-organism champion score.

The main contributions are:

1. a prospectively frozen, content-addressed G0→G1→G2 lineage in which the search policy is an executable artifact rather than a post-hoc description;
2. a replay-to-online protocol that uses public historical discovery trees for policy development while reserving fresh holdout worlds and a fixed external evaluator for validation;
3. retained negative evidence showing two distinct failure modes before the successful second transition: search-space saturation and replay overfitting;
4. a two-stage validation of G2: strict prospective replay generalization followed by a real-organism process comparison;
5. an explicit claim boundary separating process efficiency from recursive task-quality uplift, transfer, autonomous model-only authorship, and open-ended RSI.

The design is influenced directly by Dream-RSI’s observation that a completed discovery tree can function as an exact replay simulator over the search that actually occurred [8]. Genesis differs in emphasis: the goal here is not to maximize discovery benchmark performance, but to construct a conservative evidence chain in which failed recursive attempts remain part of the scientific record and later protocol changes cannot retroactively upgrade them.

2 Terminology and claim levels

The phrase recursive self-improvement is overloaded. Recent surveys distinguish self-refinement, training-time self-improvement, evaluator improvement, and broader research-process improvement [1]. We therefore use operational labels tied to observable gates rather than treating RSI as a binary property.

2.1 Organism, search policy, and mutation operator

The *organism* is the software system whose behavior is tested by the external evaluator. A *search policy* observes a public archive of past organism variants and selects parents for new proposals. A *mutation operator* produces candidate source-code edits for the selected parent. In V18–V20 that operator is an external LLM proposer running in isolated chats. The search policy does not edit the evaluator and cannot read hidden evaluator cases.

This distinction matters for authorship. The repository records G1 and G2 as parent-authored lineage generations because the parent generation supplies the inherited search state, admissible evidence, and selection context. However, the actual source edit is proposed by an external model. We therefore describe the process as *lineage-mediated system-level self-modification*, not literal model-only self-authorship.

2.2 Frozen process utility

The real-organism process experiments use the prospectively frozen lexicographic utility

$$U = (P, Q - 953, -N),$$

where P is the number of retained champion promotions, Q is best retained public quality, and N is the number of completed hidden-evaluator observations. Promotions dominate quality gain, which dominates evaluation efficiency. Thus fewer observations matter only when the higher-priority outcomes tie.

This ordering prevents a policy from winning merely by doing nothing when a competing policy discovers a genuinely better champion. It also makes the present result easy to interpret: both successful transitions occur because promotion and quality tie and the child spends fewer hidden observations.

2.3 Project levels

The evidence ledger uses the following project-specific hierarchy:

- **L0**: reproducible organism evaluation infrastructure;
- **L1**: prospective offline improvement of the search/meta-search process;
- **L2**: prospective real-organism process improvement for one parent-child transition;
- **L3-P**: two successive process-improvement transitions, with a separately validated G2 authored from G1 lineage state;
- **L4**: transfer to a materially new evaluator, organism family, or domain.

The suffix P is intentional. *L3-P* is *process* recursion. It is not presented as a community-standard RSI level and is not equivalent to equal-budget recursive quality uplift.

3 Experimental architecture

3.1 Persistent discovery archive

Genesis stores evaluated organism variants in a persistent content-addressed archive. A node records its parent, lineage depth, public quality, whether it was ever champion, a research signature describing the mutation family and mechanisms, and the source archive needed for exact replay. Non-champion but viable nodes remain available as stepping stones.

The archive design draws on open-ended evolutionary systems such as the Darwin Gödel Machine, which preserve multiple branches rather than greedily replacing a single incumbent [7]. Genesis adds a stricter separation between archive viability and champion promotion. A child can remain useful for future exploration without displacing the current champion.

3.2 Replay worlds

A replay world exposes only the portion of a previously recorded discovery tree that a policy could have known at each step. Selecting a historical parent either reveals its recorded child or consumes a represented request when no continuation was recorded. The policy never receives an oracle for unexecuted mutations. Replay therefore simulates allocation decisions over observed history rather than predicting counterfactual organism outcomes.

This matches the central Dream-RSI insight: historical discovery trees can provide low-cost feedback for alternative exploration policies because realized outcomes are already known [8]. Genesis uses that idea conservatively. Replay may select a search policy for later deployment, but real organism retention is still decided by the external evaluator.

3.3 External organism evaluator

The hidden evaluator is fixed for the real-organism epoch. It executes candidate software in a sandbox, checks hard safeguards and behavior contracts, computes capability scores and quality, and decides whether a candidate is archive-viable or champion-eligible. Its implementation and hidden cases are outside the editable organism and policy surfaces.

3.4 Prospective protocol discipline

The experiments use four recurring controls:

1. **freeze before observation**: success rules, policies, hashes and seed construction are committed before the relevant hidden or holdout result exists;
2. **one-shot evaluation**: a completed scientific result cannot be rerolled under the same protocol;
3. **negative evidence retention**: failed attempts remain in the evidence register and may motivate a new protocol but cannot be reclassified;
4. **content addressing**: policies, proposal patches, manifests and result records are bound by SHA-256 digests.

These controls do not make the project independent of its author, but they reduce the space for post-hoc selection within the recorded experimental line.

4 V14: creating G1 by prospective offline meta-improvement

The lineage used in the later real-organism experiments originates in V14. Its protocol is frozen before the prospective benchmark seed exists. The meta-search operates over a bounded parameterized family of exploration policies and includes the incumbent among candidates.

Across 24 prospectively generated replay worlds, V14 records one strict future meta-policy win, no future losses, 21 ties, and one adopted transition. The adopted transition improves future core replay value from 681 to 691. The resulting task policy changes one operational parameter that later becomes scientifically important: `stall_rounds` falls from 3 in G0 to 1 in G1.

The content-addressed generations are:

- G0 generation digest `52a928eb...c81c`, with `stall_rounds=3`;
- G1 generation digest `8e6b591f...e4e8b`, with `stall_rounds=1`.

V14 establishes only L1. It does not yet show that the shorter stall rule improves the real organism search process. The protocol explicitly reserves that question for a later hidden-evaluator comparison.

This point is methodologically important. A meta-policy can look better under replay while failing online because historical trees cover only realized edges. The project therefore treats replay as a development and policy-selection instrument rather than a substitute for real-organism evaluation.

The V14 result also does not show multi-step offline recursion: the recorded recursive-offline diagnostic is negative. We retain that fact rather than treating the later real result as if V14 had already demonstrated a recursive chain.

5 V15.4: first real-organism process improvement

V15.2 defines the real-process comparison between G0 and G1, and V15.3/V15.4 instantiate the proposer runtime and second G0 round. Both arms begin from the same A012 state with champion quality 953. Round one uses the same two parent nodes and the same accepted proposals for both arms, but each proposal is physically evaluated separately per arm.

The paired public projections are bit-identical. One proposal raises conversation behavior but introduces regressions and is not archive-viable; the other is archive-viable but neutral at quality 953. After the deterministic round merge, both arms therefore remain at the same champion and quality.

The policies now diverge only through their frozen stopping rule. G1 has `stall_rounds=1` and stops after the neutral first round, consuming two observations. G0 has `stall_rounds=3` and continues. Its round-two parent plan is frozen from public evidence before later proposals and hidden evaluations. Two further physical evaluations are both neutral at quality 953.

The terminal states are

$$U(G0) = (0, 0, -4), \quad U(G1) = (0, 0, -2).$$

Because the first two utility components tie, G1 is strictly better on scientific-observation cost. V15.4 therefore establishes the project’s L2 process-improvement criterion for $G0 \rightarrow G1$.

This is an efficiency result. G1 has not discovered a better organism than G0. Its improvement is that the inherited stopping policy avoids two evaluations that, in this realized state, would not improve the champion. A reviewer who requires equal evaluation budgets or a higher final task score should not count V15.4 as recursive quality improvement. We agree with that distinction and preserve it throughout the paper.

6 Negative recursive attempts: V16–V18

A central reason to publish the full sequence rather than only V19–V20 is that the failed recursive attempts expose the failure modes that the successful protocol eventually addresses.

6.1 V16 and V17: bounded policy-class saturation

After the $G0 \rightarrow G1$ L2 result, V16 attempts to author G2 within the inherited bounded policy family. It fails to produce a successor that passes the required gate. V17 then increases chronological replay resolution by rebuilding 13 prefix worlds from the real archive. The result remains negative. The evidence register concludes that the current bounded meta-policy family is saturated on the admitted real tree.

This is not treated as almost G2. It is retained as evidence that a narrow parameter neighborhood can exhaust its useful variation even when the broader recursive question remains open.

6.2 V18: executable-policy expansion and an overfitting failure

V18 expands the editable object from the bounded parameter vector to executable policy code, subject to an AST/ABI sandbox. Four independent isolated proposer attempts all discover the same public strategy: on the 13 real prefix worlds, stop immediately at the already-observed high-quality plateau. Each child improves mean public replay core from 943 to 953 and reduces represented requests from 1 to 0. All four are decision-equivalent despite different source hashes.

The selected source is frozen before a fresh 24-world holdout is allocated. The result is completely neutral:

0 wins, 0 losses, 24 ties.

Parent and child both obtain mean core 666.125 and mean represented requests 5.0. V18 therefore fails.

The failure is informative. A single real replay regime made a plateau-specific shortcut look universally useful. The external holdout prevents that shortcut from being admitted as G2. V19 is designed only after this negative is public.

6.3 Why V18 matters for interpretation

RRSI independently highlights a closely related concern: agent-harness evolution can show large in-distribution improvements that shrink outside the evolve set [4]. V18 is a concrete instance of that risk in Genesis. The project does not claim that V19 eliminates overfitting in general; instead, V19 expands the development distribution and reserves a new untouched holdout.

7 V19: stratified development and prospective G2 generalization

V19 keeps executable policy code as the editable target but changes the development gate before any new proposer output. The public development corpus becomes stratified:

- 13 chronological real A016-prefix replay worlds;
- 188 synthetic worlds drawn from previously public calibration, prospective and negative sequences.

The total development corpus is 201 worlds. Direct access to `view["world_id"]` is forbidden so that a policy cannot branch on training-world identity.

A candidate is eligible only if it preserves the real stratum with no paired losses while strictly improving the synthetic stratum without increasing represented requests. This prospective condition makes the V18 plateau-only shortcut insufficient.

Four independent GPT-5.6 Sol / High isolated proposer attempts are returned before laboratory feedback. All four pass the mechanical contract and are decision-equivalent across all 201 development worlds:

- real stratum: 13 wins, 0 losses, mean core 953 versus 943, requests 0 versus 1;

- synthetic stratum: 188 wins, 0 losses, mean core 750.7074 versus 673.1596, requests 2.7074 versus 3.9574. Because their behavior is identical, the frozen lexical source-hash tie break selects one canonical child, SHA-256 69d0d725...5fdf. The child source is committed before the holdout wrapper is created.

7.1 Fresh 24-world holdout

The one-shot holdout seed is derived from a future GitHub Actions run ID, the frozen child source hash, and the source-freeze commit. The seed is prospectively unavailable at source freeze; it is not claimed to be a cryptographic randomness beacon.

On the 24 fresh worlds, G2 strictly beats G1 on every world:

Metric	G1	G2	Delta
Mean core replay value	677.0833	752.3333	+75.25
Mean represented requests	3.6667	2.6250	-1.0417
Mean quality gain	48.4167	102.9583	+54.5416
Champion hits	12	24	+12

Paired outcome: 24 wins, 0 losses, 0 ties.

The holdout passes the frozen offline gate and admits G2 with generation digest d2e2b990...c6bd. This is stronger evidence than V18 because the child generalizes beyond the public stratified corpus. It still does not establish the real-organism G1→G2 transition.

8 V20: second real-organism transition

V20 freezes the real G1→G2 comparison after V19 passes and before any V20 proposer output or hidden-organism observation. Both policies are applied to the same reconstructed A016 public state, whose retained champion quality is 953.

8.1 First-wave policy decisions

The policies make different prospectively frozen decisions:

- G1 selects two parents, `v8-node-015-913198e1c00a` and `v8-node-014-2334b1dcc01a`;
- G2 returns an empty parent batch and stops before hidden evaluation.

The empty batch is explicitly allowed by the V20 protocol. The laboratory is forbidden to force G2 to spend an observation merely to equalize budget.

For each requested G1 slot, three fresh isolated proposer attempts are generated. All six attempts are valid. The fixed first-valid rule accepts attempt 1 in each slot, so no later content is selected based on evaluator outcome.

8.2 Two physical G1 observations

Each accepted proposal is evaluated exactly once from an independent exact A016 copy using the same hidden evaluator authority. Both results are identical on the facts relevant to terminal utility:

- hard pass: true;
- archive viable: true;
- quality: 953;
- improvements: none;
- regressions: none;
- champion promotion: false.

Under `stall_rounds=1`, G1 terminates after this neutral first wave. Terminal utilities are

$$U(G1) = (0, 0, -2), \quad U(G2) = (0, 0, 0).$$

G2 is therefore strictly better under the frozen lexicographic utility.

8.3 *L3-P* verdict

The project-defined *L3-P* condition requires:

1. an established $G0 \rightarrow G1$ real-process improvement;
2. a $G1$ -lineage $G2$ that passes a separately frozen prospective generalization gate;
3. an established $G1 \rightarrow G2$ real-process improvement.

V15.4, V19 and V20 satisfy these three conditions respectively. The resulting chain is therefore recorded as *L3-P PASS*.

The result should be read literally. $G2$ is a better search process for the measured state because it avoids evaluations that $G1$ performs without benefit. The real-organism champion remains 953. No recursive increase in champion quality has been shown.

9 Post-result hostile audit and reproducibility

The project performed an explicit post-result adversarial review after V20. This review is AI-assisted internal self-audit, not independent peer review.

9.1 Independent recomputation path

A post-result script separate from the V20 terminal finalizer regenerates the V19 holdout from its frozen seed construction, reloads the exact $G1$ and $G2$ sources, recomputes the 24-world comparison, reproduces the fully revealed A016 first-wave decisions, rereads the two immutable V20 physical result records, and recomputes the terminal utilities.

That path recovers:

- V19 holdout: 24 wins, 0 losses, 0 ties, mean core delta +75.25, mean request delta -1.0417;
- A016 decisions: $G1$ selects the two frozen parents; $G2$ selects none;
- V20 utilities: $G1=(0,0,-2)$, $G2=(0,0,0)$.

A separate direct cross-check rereads all six V15 physical observation records. The two round-one $G0/G1$ public projections match pairwise, the two $G0$ round-two results are neutral at 953, and the recomputed first-transition utilities are $G0=(0,0,-4)$, $G1=(0,0,-2)$.

The successful V19/V20 audit is preserved as a CI-produced result. A later wrapper intended to combine V15 and V19/V20 into one GitHub Actions job failed before runner assignment during the observed reruns; it is not represented as a successful scientific execution. The direct V15 cross-check is recorded separately.

9.2 Reproducibility versus replication

The repository makes internal reproduction unusually explicit: protocol commits, bundle hashes, source freezes, holdout seed construction, exact executable policies and physical result records are public. A reader can reconstruct the logic from preserved artifacts.

This is still not an independent replication. The project author operated the experimental workflow, and the mutation candidates were produced in manually created isolated chats. Independent execution by an unaffiliated group remains a stronger unmet standard.

9.3 Why negative results remain load-bearing

The audit verifies not only the final positive chain but also the preserved V18 holdout failure. This is important because V19's design is a response to that specific failure. Removing V18 from the narrative would make the stratified V19 corpus look arbitrary and would hide the strongest evidence that the holdout gate can reject an apparently successful self-modification.

10 Related work

10.1 Empirical self-modifying agents

The Darwin Gödel Machine replaces formal proof of beneficial self-modification with empirical evaluation and preserves an archive of diverse agents [7]. Gödel Agent exposes agent logic to iterative self-modification under high-level objectives [5]. The Self-Taught Optimizer treats an improver program as an optimization target [6]. These systems establish the practical importance of modifying the machinery around a base model rather than only prompting a fixed agent.

Genesis shares the empirical, system-level view but places unusual weight on prospective protocol ordering and negative-evidence retention. Its central positive result is also narrower: process efficiency under a frozen evaluator, not open-ended benchmark improvement.

10.2 Exploration policy evolution and replay

Dream-RSI is the closest methodological relative. It treats accumulated discovery trees as exact replay simulators over realized search and recursively improves executable exploration policies offline before redeployment [8]. Genesis adopts the same central insight but couples it to a separate hidden organism evaluator and to one-shot prospective gates. V18 demonstrates why this matters: a policy can dominate a public replay regime while failing completely to generalize.

10.3 Comparative and group evolution

Group-Evolving Agents addresses information isolation between tree branches by sharing experience across groups [3]. Mendel Gödel Machine similarly argues that multi-trajectory and cross-lineage evidence improve mutation quality [2]. Earlier Genesis archive designs also preserve cross-lineage evidence, but the V14–V20 chain studied here keeps the central causal variable on the search-policy side.

10.4 Overfitting and regularization

RRSI explicitly studies overfitting in recursively evolved agent harnesses and adds proposal- and selection-side regularization to improve out-of-distribution performance [4]. Genesis V18 exhibits the same broad hazard in a smaller controlled setting. V19 addresses it through stratified development, a world-identity ban and a fresh holdout rather than through RRSI’s critic/pruner architecture.

10.5 Taxonomies of RSI

Recent surveys emphasize that self-improvement spans behavior, policy, evaluator and research-process changes, and that verification strength is central to credible claims [1]. This paper’s *L3-P* label is intended to make that distinction operational: the measured recursion is in the exploration process under fixed external evaluation, not in the evaluator or base model weights.

11 Limitations and alternative explanations

11.1 The gain is process efficiency, not quality uplift

The most important limitation is also the simplest. Both real-process transitions tie on champion promotions and retained quality. The child wins because it spends fewer hidden observations. A reader who defines RSI only as recursively increasing task quality should regard the present result as evidence about recursive search-process efficiency, not about that stronger definition.

11.2 G2 contains a development-adapted stopping threshold

The selected G2 stops when observed best quality is at least 780. This threshold was produced from the V19 public development corpus. Direct access to world identity was forbidden, and the fresh V19 holdout strongly rejects pure memorization of the 201 development worlds, but the threshold is still development-adapted. V20 then evaluates one real state whose public quality is 953. The result does not show that the same stopping rule transfers to unrelated organism families.

11.3 One real terminal state

V20 has two independent physical G1 child evaluations, but the process comparison begins from one real A016 state. This supports a causal state-level claim, not a population estimate. Multiple fresh campaigns are needed before making frequency or robustness claims about the real-process advantage.

11.4 Repeated use of one evaluator epoch

The hidden evaluator implementation remains fixed and outside the editable system, but prior aggregate results become public over the course of the project. This can create adaptive pressure against one evaluation authority even when hidden cases are never revealed. A strong L4 experiment should use a new evaluator/task family that was not part of V14–V20 development.

11.5 External LLM mutation operator

G2 is not written solely by the prior policy. An external LLM supplies candidate source edits from the frozen bundle. The lineage controls what evidence and search state are exposed and which candidate survives, but the mutation operator itself is externally provided. This is comparable to evolutionary systems that use a foundation model as a mutation engine, but weaker than a fully autonomous closed loop.

11.6 Manual proposer isolation

Temporary-chat isolation is procedural. Hashes and transcript schemas prove which bundle and output artifacts entered the laboratory, but they cannot cryptographically prove that the operator selected the specified model/effort or that no outside context entered a chat. Future work should replace this with a provider-attested or locally isolated proposer runner.

11.7 Project-defined $L3-P$

The term $L3-P$ is defined by this project. It should not be read as a consensus field-wide RSI level. We use it to prevent a bounded process result from being conflated with stronger concepts such as unbounded recursive intelligence growth.

11.8 No statistical population inference

The key gates are deterministic or paired one-shot protocol tests, not samples designed for population-level statistics. The 24-world holdout is useful evidence of generalization over that generator, but it does not justify a claim that G2 will win on essentially all possible search worlds. The paper reports exact observed counts rather than confidence intervals implying a broader sampling frame.

12 Discussion and conclusion

The most useful interpretation of the Genesis IV result is not that a software organism became broadly more intelligent twice. It is that an improvement mechanism became measurably more selective about when expensive search was worth continuing, and that this selectivity itself survived a second recursive transition.

The path to that result was not monotone. A bounded parameter family first produced G1. Attempts to recurse within that family saturated. Expanding to executable policy code then produced an attractive but brittle shortcut that failed every fresh V18 holdout comparison by tying the parent. Only after the development distribution was diversified did a G2 policy emerge that generalized prospectively. That history suggests a practical lesson shared with recent work on harness regularization: recursive systems need explicit mechanisms against development-set exploitation, and failed recursive generations are often more informative than an uninterrupted sequence of wins.

The V20 result also gives a concrete example of why process utility deserves separate measurement. In many expensive discovery systems, knowing when *not* to evaluate is itself valuable. G2’s advantage is abstention: on A016 it stops because the observed state already exceeds its learned threshold. The control policy continues, requests two plausible mutations, and the hidden evaluator confirms that neither improves the champion. Under the frozen utility, the abstaining policy is therefore better.

Whether this kind of efficiency recursion can lead to recursive quality uplift is unknown. One possibility is that a better search policy first saves budget on saturated regions and later reallocates that budget to higher-value branches. Dream-RSI reports related adaptive changes in exploration breadth and cost over recursive rounds [8]. Demonstrating that stronger effect in Genesis would require a new prospective protocol in which later generations actually discover higher-quality real-organism champions under an equal or otherwise controlled budget.

The next scientifically meaningful targets are:

1. repeat the $G1 \rightarrow G2$ comparison from multiple fresh real campaign states;
2. test the frozen $G2$ policy under a new evaluator/task family;
3. automate and attest proposer isolation;
4. define an equal-budget recursive-quality gate separate from $L3-P$;
5. attempt $G2 \rightarrow G3$ only under a protocol that cannot redefine the already established claim.

Conclusion. Genesis IV establishes a prospectively validated, bounded two-transition recursive *process*-improvement chain. $G0 \rightarrow G1$ reduces hidden-evaluation cost from four to two observations at unchanged retained quality. A lineage-mediated $G2$ then survives a fresh 24-world replay holdout with 24 paired wins and, on the real A016 state, matches $G1$ ’s retained outcome while requesting zero hidden evaluations rather than two. This satisfies the project’s preregistered $L3-P$ definition. It does not establish open-ended RSI, general transfer, or recursive quality uplift.

AI-assistance disclosure

Generative-AI systems were used extensively during software development, proposer execution, adversarial review, data inspection, and manuscript preparation. Anthony Mets remains responsible for research objectives, protocol decisions, acceptance or rejection of generated work, scientific claims, and publication. AI-assisted internal review is not represented as independent peer review or external replication.

Artifact availability

The publication package is maintained at <https://github.com/mjodheim/mira-genesis>. The frozen experimental evidence for V14–V20 is preserved in the public `mjodheim/mira` repository on the `research/genesis-dream-rep` line, including preregistrations, source freezes, holdout records, immutable physical-evaluation branches, the evidence register, and post-result audit artifacts.

References

- [1] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in ai: From bounded self-refinement to autonomous research loops. *arXiv preprint arXiv:2607.07663*, 2026.
- [2] Changzhi Liu, Yilun Liu, Sikuan Yan, Volker Tresp, and Yunpu Ma. Mendel gödel machine: Recursive self-improving coding agents via comparative evolution. *arXiv preprint arXiv:2608.07645*, 2026.
- [3] Zhaotian Weng, Antonis Antoniadis, Deepak Nathani, Zhen Zhang, Xiao Pu, and Xin Eric Wang. Group-evolving agents: Open-ended self-improvement via experience sharing. *arXiv preprint arXiv:2602.04837*, 2026.
- [4] Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. Rrsi: Regularized recursive self-improvement of agent harnesses. *arXiv preprint arXiv:2609.24972*, 2026.
- [5] Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.
- [6] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. *arXiv preprint arXiv:2310.02304*, 2023.
- [7] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.
- [8] Tong Zheng, Xidong Wu, Zheng Zhang, Zhankui He, Chaoyi Zhang, Benjamin Coleman, Ruoqiao Wei, Di Bai, Haolin Liu, Rui Liu, Xue Wang, Yue Zhuan, Wang-Cheng Kang, Renkai Xiang, Heng Huang, Xinwu Cheng, and Yunsong Guo. Dream-rsi: Recursive self-improvement through evolving worlds. *arXiv preprint arXiv:2609.14858*, 2026.